



# **Module 6: Hashing**

# Module 6: Hashing

- Hash functions

- Open Hashing( Closed Addressing)

- Separate Chaining

- Closed Hashing(Open Addressing)

- Linear Probing
    - Quadratic Probing
    - Double Hashing

- Random Probing

- Rehashing

- Extendible Hashing

# Hashing

- Implementation of hash table is called Hashing
- Hashing is a technique used for performing insertions, deletions, and find in constant average time

# Goals of Hash- ADT

- Structure will be a table (relatively small)
- Table completely contained in memory
- Implemented by an array
- Capitalizes on ability to access any element of the array in **constant** time

# Hashing- General Idea

- Hash table data structure is an **array of some fixed size** containing the items( Keys and associated values)
- For Example: Generally Search is performed on data members with respect to the item.  
**This is called the key.**

# HASH Table

Table Size : 0 to Tablesize-1

| Keys | Associated Values |
|------|-------------------|
| 0    |                   |
| 1    |                   |
| 2    |                   |
| 3    | Jack              |
| 4    | Priya             |
| 5    |                   |
| 6    | Dinesh            |
| 7    | Martin            |
| 8    |                   |
| 9    | Suban             |

Each key is mapped into some number in the range **0 to TableSize - 1** and placed in the appropriate cell.- **Hash Function**

# Problems in Hash Table ADT

- Choosing a function
- Deciding the Hash Table Size
- Deciding what to do when two keys hash to the same value(Also called as Collision)

# Hash Functions

- A hash function is a key to address transformation
- Determines position of key in the array
- **For Example:** If the input keys are integers, then simply returning **Key mod TableSize** is generally a reasonable strategy

**Key % Table size**



# Hash Functions

- Let us consider  $f(x) = x \% 15$ . Then,

|        |   |    |     |    |    |    |    |
|--------|---|----|-----|----|----|----|----|
| $x$    | = | 25 | 129 | 35 | 26 | 47 | 36 |
| $f(x)$ | = | 10 | 9   | 5  | 11 | 2  | 6  |

- Storing the keys in the array is straightforward:

| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|---|----|---|---|----|----|---|---|-----|----|----|----|----|----|
|   |   | 47 |   |   | 35 | 36 |   |   | 129 | 25 | 26 |    |    |    |

- Thus, **Delete** and **FIND** can be done in  $O(1)$ , and also insert,

But Except??????????

# Hash Functions

- What happens when you try to insert:  $x = 65$  ?

$$x = 65$$

$$f(x) = 5$$

| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|---|----|---|---|----|----|---|---|-----|----|----|----|----|----|
|   |   | 47 |   |   | 35 | 36 |   |   | 129 | 25 | 26 |    |    |    |
|   |   |    |   |   | 65 |    |   |   |     |    |    |    |    |    |

This is called a **collision**.

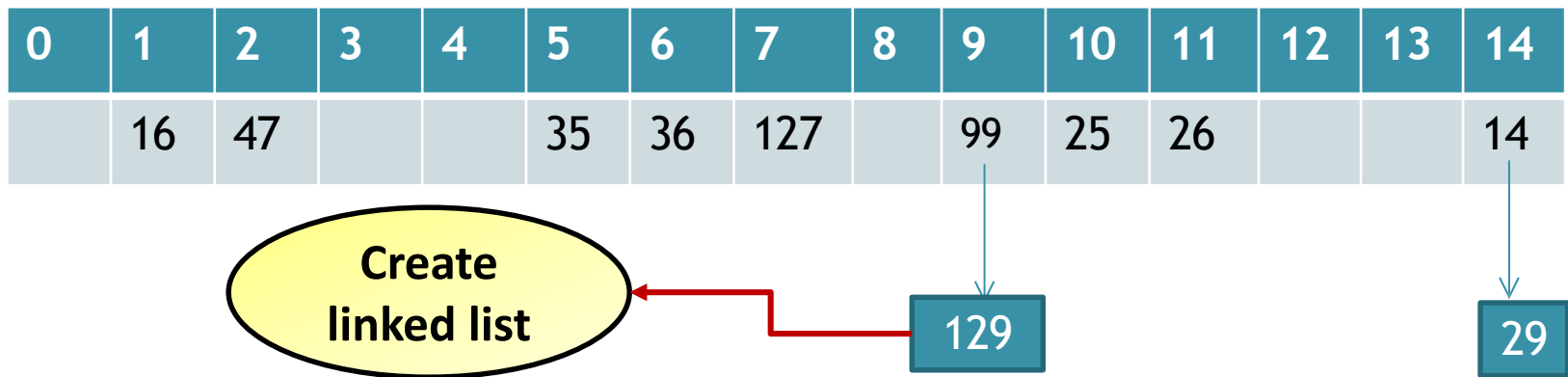
# Handling Collisions

- When an element is inserted, it hashes to the same value as an already inserted element, then we have a collision and need to resolve it.
- Types of Hashing
  - Open Hashing( ClosedAddressing)
    - Separate Chaining
  - Closed Hashing(OpenAddressing)
    - Linear Probing
    - Quadratic Probing
    - Double Hashing

# Open Hashing - Separate Chaining

- Let us consider where the following elements stored in **16,14,29,99,129**
- Let us consider  $f(x) = x \% 15$ . Then,

|          |           |           |           |           |            |
|----------|-----------|-----------|-----------|-----------|------------|
| <b>X</b> | <b>14</b> | <b>16</b> | <b>29</b> | <b>99</b> | <b>129</b> |
| $f(x)$   | 14        | 1         | 14        | 9         | 9          |



- New keys go at the front of the respective chain.

# Separate Chaining: Disadvantages

- Parts of the array may never be used.
- As chains get longer, search time increases to  $O(n)$  in the worst case.
- Constructing new chain nodes is relatively expensive (still constant time, but the constant is high).
- Is there a way to use the “unused” space in the array instead of using chains to make more space?

# Closed Hashing(Open Addressing)

- Linear Probing
- Let Key  $x$  be stored in element  $f(x)=t$  of the array

| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|---|----|---|---|----|----|---|---|-----|----|----|----|----|----|
|   |   | 47 |   |   | 35 | 36 |   |   | 129 | 25 | 26 |    |    |    |

65 ??

- If the hash table is not full - Attempt to store in the next array element

Example:  $(t+1)\%N, (t+2)\%N, (t+3)\%N \dots\dots$   
*Untill finding an empty slot*

# Linear Probing

- For Example: Where do you store 65?

$$65\%15=5$$

| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|---|----|---|---|----|----|----|---|-----|----|----|----|----|----|
|   |   | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 26 |    |    |    |

Diagram illustrating the storage of 65 in a hash table using linear probing. The hash table has 15 slots (0 to 14). The value 65 is stored at index 7. The number of probes required is 3, indicated by the circled numbers 1, 2, and 3 below the indices 5, 6, and 7 respectively, with arrows pointing to the corresponding slots.

- For Example :Where do you store 16 ?

$$16\%15=1$$

| 0 | 1  | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|----|----|---|---|----|----|----|---|-----|----|----|----|----|----|
|   | 16 | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 26 |    |    |    |

Diagram illustrating the storage of 16 in a hash table using linear probing. The hash table has 15 slots (0 to 14). The value 16 is stored at index 1. The number of probes required is 1, indicated by the circled number 1 below the index 1, with an arrow pointing to the corresponding slot.

# Linear Probing

- For Example: Where do you store 29 ?

$$29 \% 15 = 14$$

| 0 | 1  | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|----|----|---|---|----|----|----|---|-----|----|----|----|----|----|
|   | 16 | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 26 |    |    | 29 |

Number of  
Probes: 1

1

- For Example: Where do you store 14 ?

$$14 \% 15 = 14$$

| 0  | 1  | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|----|----|----|---|---|----|----|----|---|-----|----|----|----|----|----|
| 14 | 16 | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 26 |    |    | 29 |

2

Number of  
Probes: 2

1



# Linear Probing

- For Example :Where do you store 99 ?  
 $99\%15=9$

| 0  | 1  | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|----|----|----|---|---|----|----|----|---|-----|----|----|----|----|----|
| 14 | 16 | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 26 | 99 |    | 29 |

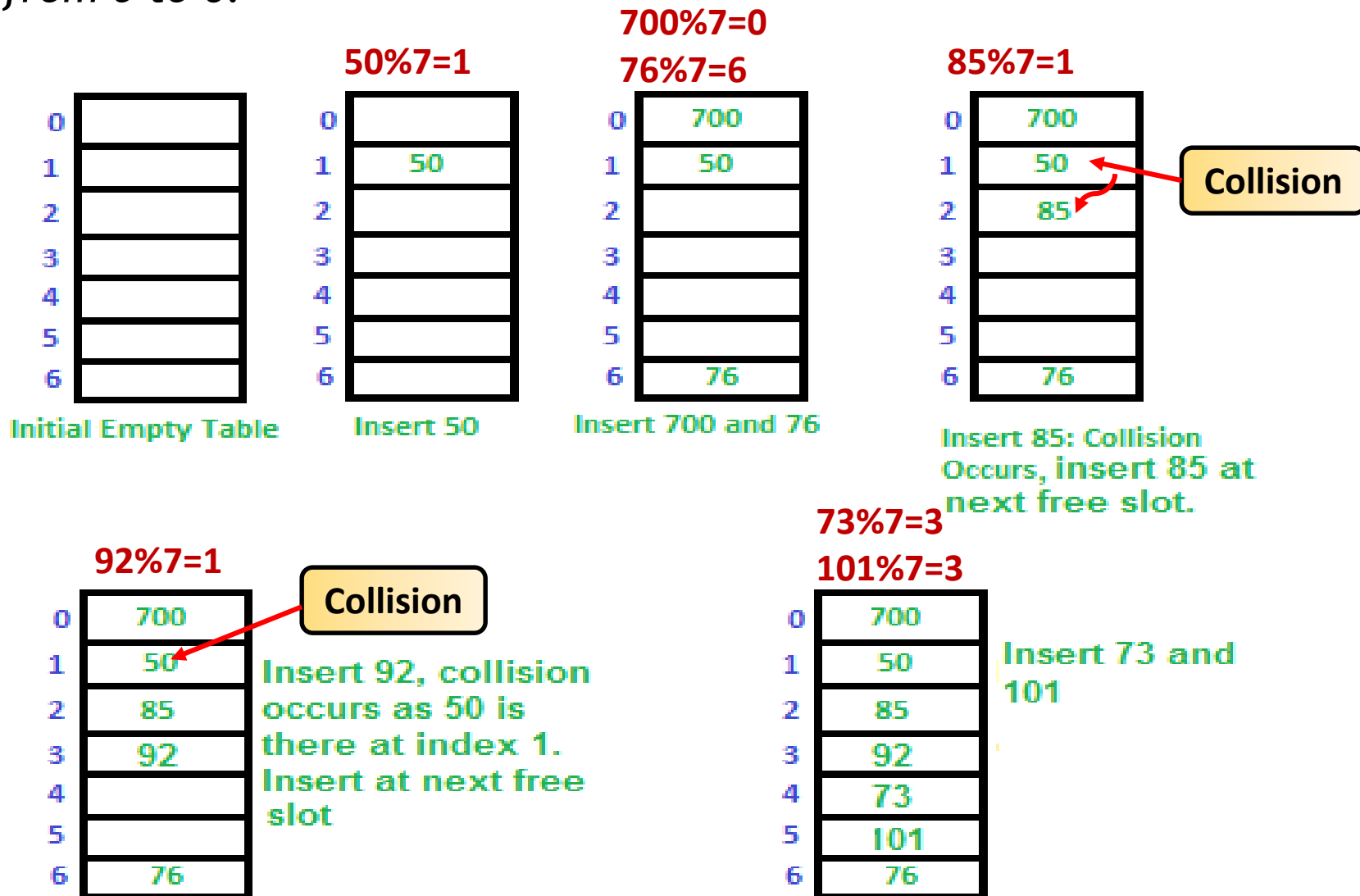
Number of Probes: 4

- For Example :Where do you store 127 ?
- $127\%15=7$

| 0  | 1  | 2  | 3 | 4 | 5  | 6  | 7  | 8   | 9   | 10 | 11 | 12 | 13 | 14 |
|----|----|----|---|---|----|----|----|-----|-----|----|----|----|----|----|
| 14 | 16 | 47 |   |   | 35 | 36 | 65 | 127 | 129 | 25 | 26 | 99 |    | 29 |

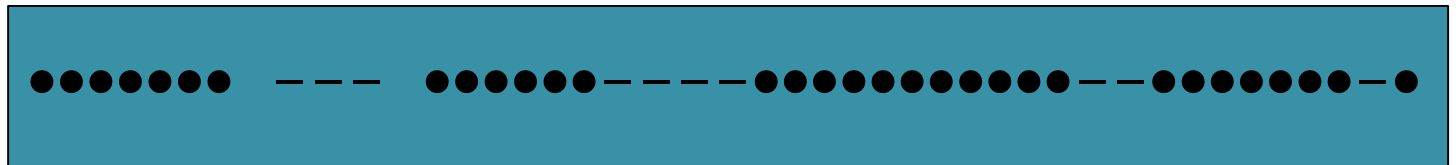
Number of Probes: 2

**Example:** Let us consider a simple hash function as “key mod 7” and a sequence of keys as 50, 700, 76, 85, 92, 73, 101, which means  $\text{hash}(\text{key}) = \text{key} \% S$ , here  $S = \text{size of the table} = 7$ , indexed from 0 to 6.



# Linear Probing

- Eliminates need for separate data structures (chains), and the cost of constructing nodes.
- Leads to problem of clustering. Elements tend to cluster in dense intervals in the array.



- Search efficiency problem remains.
- Deletion becomes trickier

# Closed Hashing(Open Addressing)

- Quadratic Probing

- Let Key  $x$  be stored in element  $f(x)=t$  of the array

| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|---|----|---|---|----|----|---|---|-----|----|----|----|----|----|
|   |   | 47 |   |   | 35 | 36 |   |   | 129 | 25 | 26 |    |    |    |

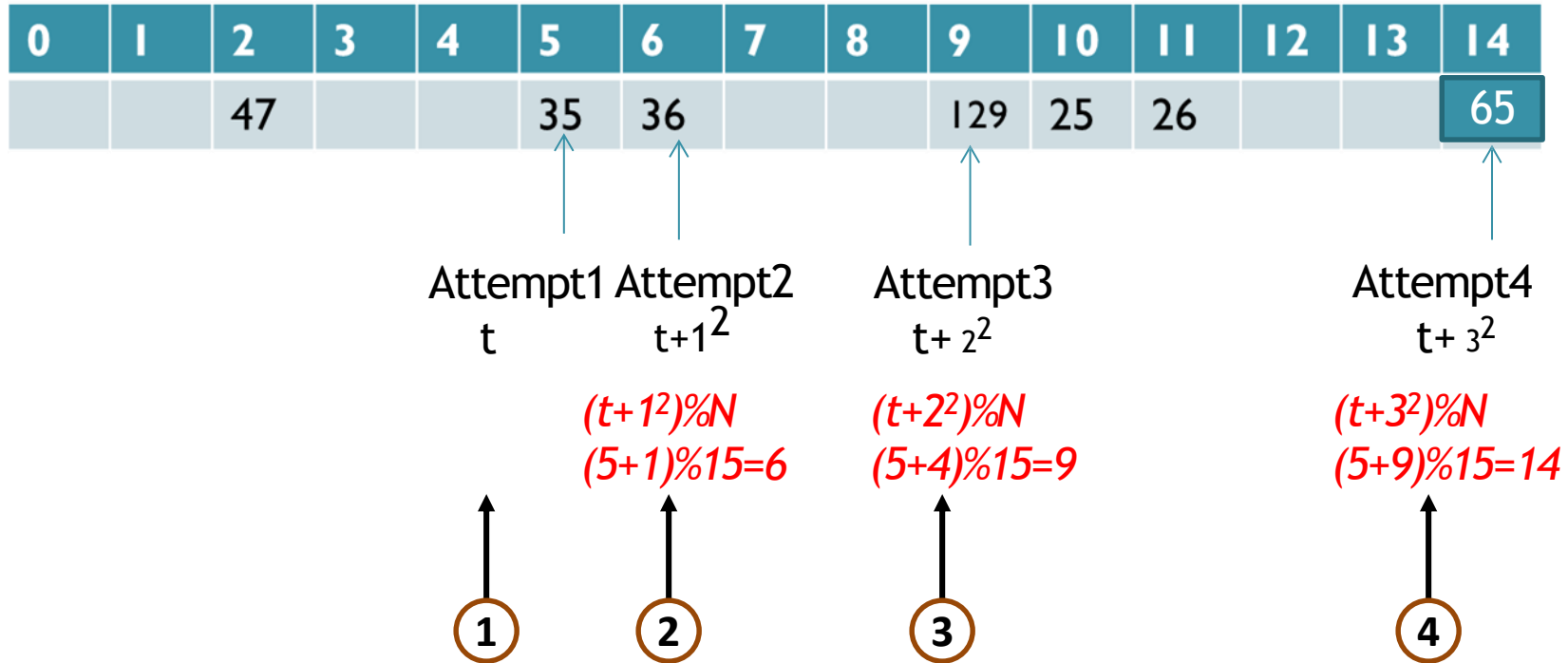
65

??

- What to do??
- If the hash table is not full - Attempt to store in the next array element( For Example:  $(t+1^2)\%N$ ,  $(t+2^2)\%N$ ,  $(t+3^2)\%N$  ... .. *Untill finding an empty slot*

# Quadratic Probing

- For Example : **Where do you store 65?**  
 **$65\%15=5$  ( $t=5$ )**



Number of Probes: 4

- For Example :Where do you store 99?  
 $99\%15=9$

| 0  | 1  | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|----|----|----|---|---|----|----|----|---|-----|----|----|----|----|----|
| 14 | 16 | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 26 | 99 |    | 29 |

- In the case of above scenario in linear probing  
 Number of Probes:4

## Quadratic Probing

| 0  | 1  | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|----|----|----|---|---|----|----|----|---|-----|----|----|----|----|----|
| 14 | 16 | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 26 |    | 99 | 29 |

Attempt1    Attempt2    Attempt3  
 t            t+1            t+4  
 $(t+1^2)\%N$      $(t+2^2)\%N$   
 $(9+1)\%15=10$      $(9+4)\%15=13$

Number of Probes:3

# Quadratic Probing

- Tends to distribute keys better than linear probing
- Resolves the problem of clustering
- Runs the risk of an infinite loop on insertion, unless precautions are taken.
  - For Example: Consider inserting the key 16 into a table of size 16, with positions 0, 1, 4 and 9 already occupied.

# Closed Hashing(Open Addressing)

- Double Hashing

- Let Key  $x$  be stored in element  $f(x)=t$  of the array

| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|---|----|---|---|----|----|---|---|-----|----|----|----|----|----|
|   |   | 47 |   |   | 35 | 36 |   |   | 129 | 25 | 26 |    |    |    |

65

??

- What to do??

- Define a second hash function  $f_2(x)=d$ . Attempt to store key in next array elements  $(t+1d)\%N$ ,  $(t+2d)\%N$ ,  $(t+3d)\%N$  ... until you find an empty slot.

Formula- $\rightarrow (H(\text{key}) + i * H_2(\text{key})) \bmod N$

- $N \rightarrow \text{Table\_size}$
- $i \rightarrow 0 \text{ to Table\_size}-1$



# Double Hashing

- The popular second hash function:

$$f_2(x) = R - (x \% R)$$

where  $R$  is a prime number,  $R < N$

# Double Hashing

- For Example : **Where do you store 65 ?**

| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|---|----|---|---|----|----|---|---|-----|----|----|----|----|----|
|   |   | 47 |   |   | 35 | 36 |   |   | 129 | 25 | 26 |    |    |    |

Probe 1

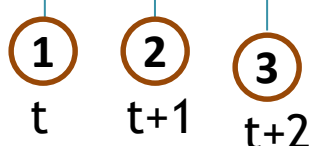
(*t*)

- As we from know Hash Function 1:  $f(x) = x \% N$   
 $f(65) = 65 \% 15 = 5$  ( $t=5$ ), **Collision Occurred**
- Apply Second Hash Function  
 $f_2(x) = 11 - (x \% 11)$
- Consider  $R = 11$  (Prime number  $< \text{Table\_size}$ )

# Double Hashing

- For Example : **Where do you store 65 ?**

| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11 | 12 | 13 | 14 |
|---|---|----|---|---|----|----|----|---|-----|----|----|----|----|----|
|   |   | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 26 |    |    |    |



No of Probes:3

*Attempt to store key in next array elements  $(t+1d)\%N$ ,  $(t+2d)\%N$ ,  $(t+3d)\%N$  ...*

- **Probe 1**  $\rightarrow f(65) = (65\%15) = t = 5$  [Collision Occurred]

In case of collision apply Second Hash Function

$$f_2(x) = 11 - (x \% 11)$$

$$f_2(65) = (11 - (65\%11)) = d = 1$$

- **Probe 2**  $\rightarrow (t+1d)\%N = (5+1*1)\%15 = 6$
- **Probe 3**  $\rightarrow (t+2d)\%N = (5+2*1)\%15 = 7$

# Rehashing

- When hash table becomes too full, running time for operations will take more time.
- **For Example:** Insertions might fail for closed hashing with quadratic resolution.
- To overcome this type of situation Rehashing technique is used.
- A solution, then, is to build another table that is about twice as big (**with an associated new hash function**) and scan down the entire original hash table, computing the new hash value for each (**non deleted**) element and inserting it in the new table.

# Rehashing

- Let us consider  $f(x) = x \% 7$  Then,

$$\begin{array}{rclclclcl} x & = & 13 & 15 & 6 & 24 \\ f(x) & = & 6 & 1 & 0 & 3 \end{array}$$

Applying  
Linear  
probing



| 0 | 1  | 2 | 3  | 4 | 5 | 6  |
|---|----|---|----|---|---|----|
| 6 | 15 |   | 24 |   |   | 13 |

For Example : if 23 is inserted into the table, the resulting table will be almost full(70%), so new table will be created

| 0 | 1  | 2  | 3  | 4 | 5 | 6  |
|---|----|----|----|---|---|----|
| 6 | 15 | 23 | 24 |   |   | 13 |

# Rehashing

- **IDEA:** The size of this table is 17, because this is the first prime that is twice as large as the old table size.
- The new hash function is then  $h(x) = x \bmod 17$ . The old table is scanned, and elements 6, 15, 23, 24, and 13 are inserted into the new table.

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|----|----|---|----|----|----|----|----|----|----|
|   |   |   |   |   |   | 6 | 23 | 24 |   |    |    |    | 13 |    | 15 |    |

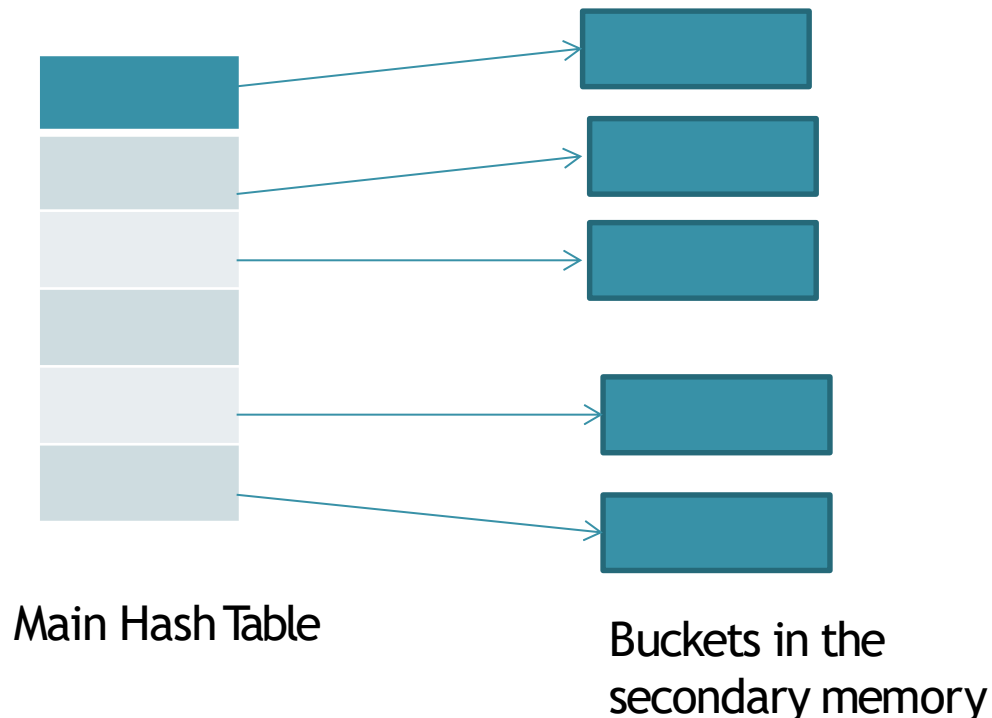
Linear probing

# Rehashing

- Go to rehashing ,
  - When table is half full.
  - When an insertion fails.
  - When table reaches load factor.
- Time Taking Operation
- Running time is  $O(N)$ , since there are  $N$  elements to rehash and the table size is roughly  $2N$

# Extendible hashing

- Main Hash table is stored in Main memory and the buckets are stored in the disk.
- Each value in the hash table is a pointer to a bucket in the secondary memory.
- Hash function is applied on the input value to generate the bucket pointer and perform the operations.



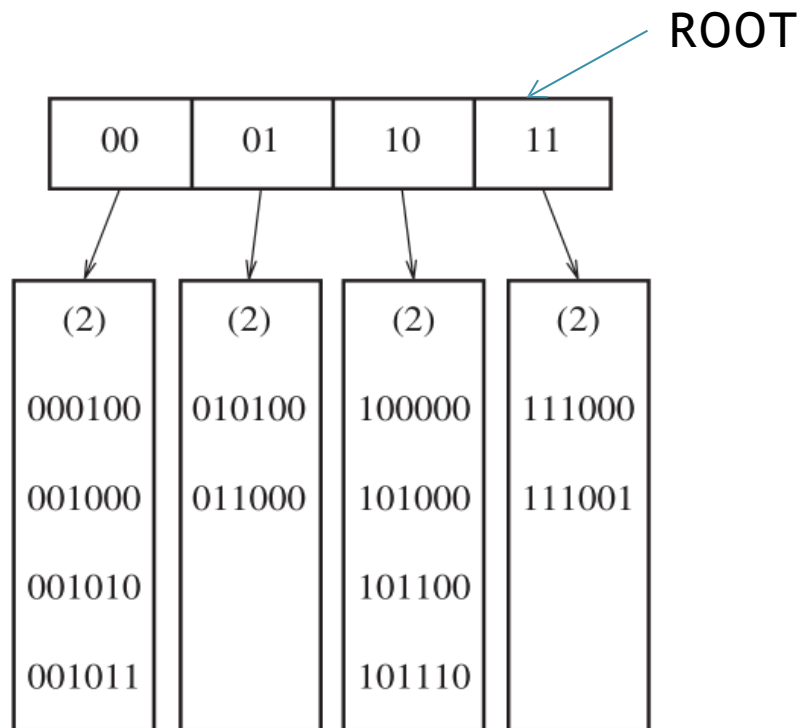


# Example

- Input our data consists of several 6-bit integers.
- To store these values??
- The root of the “tree” contains four pointers determined by the leading two bits of the data.

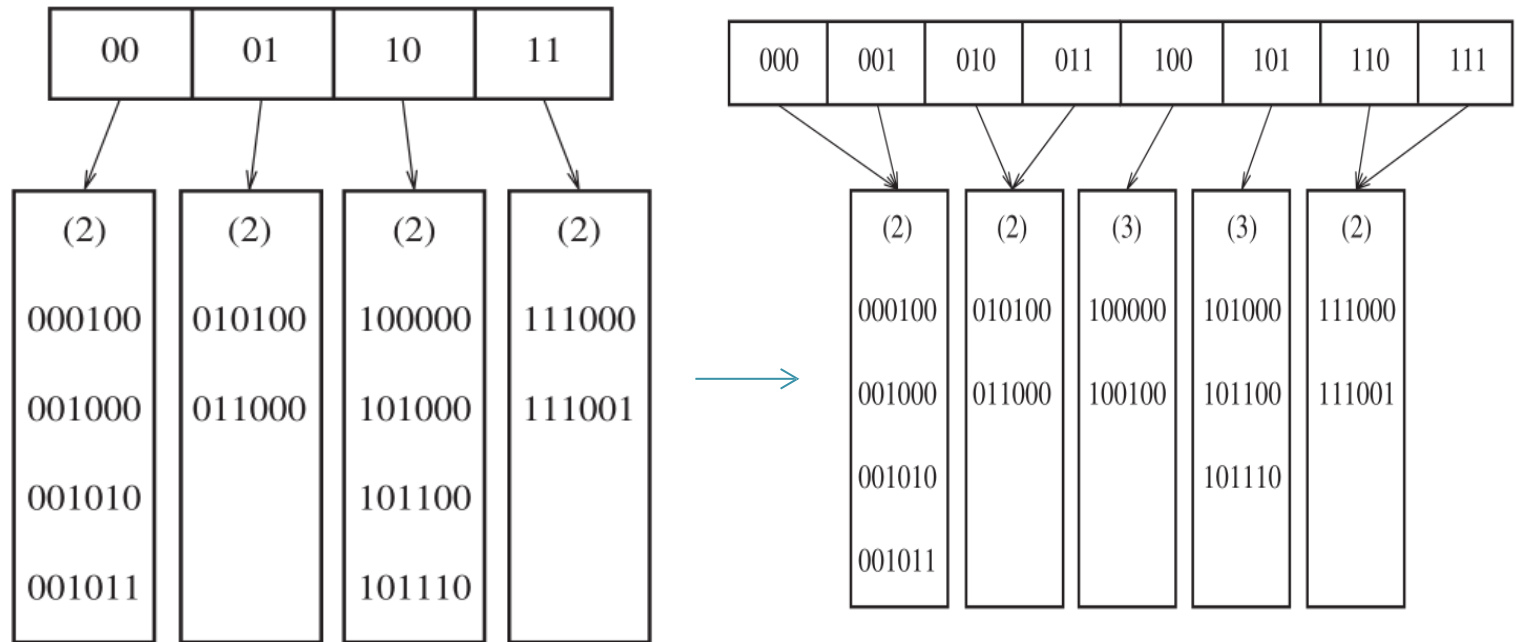
D- Number of bits used by the root. So number of entries in the leaf should be  $2^D$

Each leaf has up to 4 elements



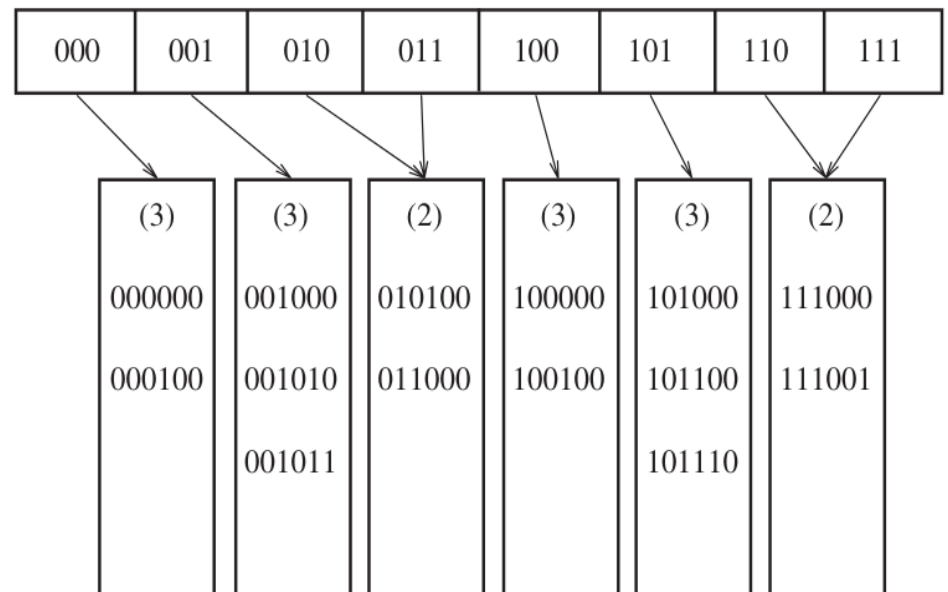
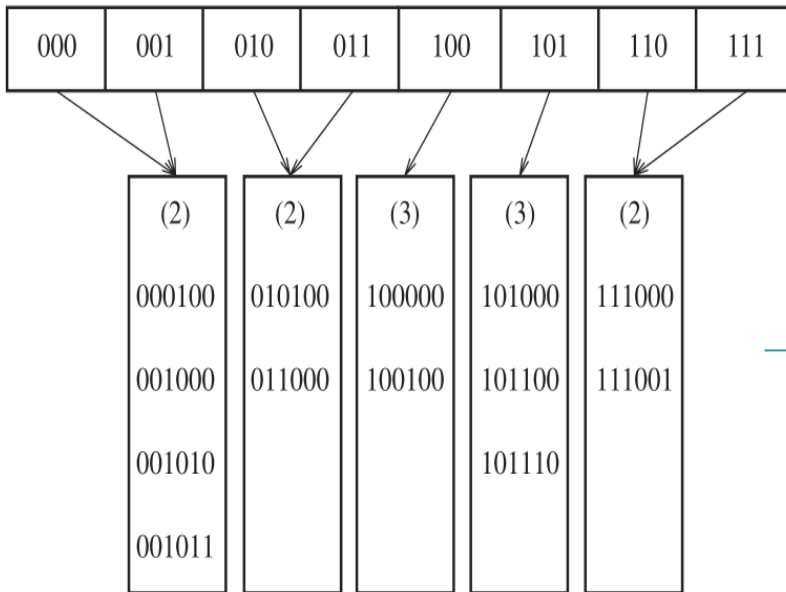
# Example

- Insert the key 100100, it points to third leaf, but it is already full, so split this leaf into two leaves, which are determined by first three bits.



# Example

- Insert the key 000000, it points to first leaf, but it is already full, so split this leaf into two leaves, which are determined by first three bits.



# Extendible hashing

- Advantages

- Simple
- Quick access time for insert and find operations in large databases.

- Disadvantages

- Not suitable when more duplicates are taken